Running the system

This file covers how to start the installation, what each script does, and what has to be switched on before anything appears on the projector.

Quick start on one machine

Double-click `start.command` in Finder. That is the whole procedure.

It checks the dependencies, repairs what is missing, opens Chrome on the main page, and prints the phone URL in the Terminal window. Leave that Terminal window open. Closing it kills the server and the visualization stops.

The system is only ever started this way. The launchers exist so that nobody has to remember what to install or which flags to pass, and every dependency check lives behind them. Starting `server.js` from a terminal skips all of it.

On a machine that has never run the system before, do the one-time step in the next section first. Otherwise macOS will refuse to open the launcher at all.

Clearing the macOS quarantine flag

macOS attaches a `com.apple.quarantine` attribute to anything that arrives from the internet, AirDrop, a messaging app, or an unzipped archive. A `.command` file carrying that flag will not open by double-click. Gatekeeper puts up a dialog saying the file cannot be verified, and the server never starts.

`preflight.sh` cannot fix this. Quarantine blocks the launcher before a single line of its code runs, so the repair has to come from outside. Do it once per machine, from Terminal, in the project folder:

```
cd /path/to/DTA2026
xattr -dr com.apple.quarantine .
chmod +x *.command *.sh
```

The first command strips the flag from every file in the folder. It exits cleanly whether or not the files were actually flagged, so it is safe to run at any time. The second restores the executable bit, which zip archives routinely drop. Without it the launchers open in a text editor instead of running.

Two notes from practice. Running `xattr -d com.apple.quarantine <file>` on a single file that was never flagged prints `No such xattr` and exits 1. That is not a failure, and the recursive form above avoids it. And if you only need to get past the dialog once, right-click the launcher and choose Open, which offers to run it anyway. That approves one file, so use the command above when setting up a machine properly.

To check the current state:

```
xattr -r -p com.apple.quarantine .
```

Silence means nothing is flagged.

You only have to do this for one launcher. Once any of them opens, `preflight.sh` clears the quarantine flag and restores the executable bit on all the others, so a machine hits the Gatekeeper dialog once rather than three times.

What has to be running

Four things, in this order:

1. **The server.** One per machine, started by a `.command` file. Port 3000.
2. **The main page** at `http://localhost:3000/`, in Google Chrome. The launcher opens it for you.
3. **The controls popup.** `main.js` opens `controls.html` in a second window automatically. Chrome blocks popups on first run, so you have to allow them for `localhost:3000` and reload. Without the popup you have no parameter controls.
4. **The phone controller**, only if you want it. Open `http://<LAN_IP>:3000/phone.html` on a phone connected to the same network. The controls popup shows a QR code for this address.

Chrome is required, not a preference. Pose detection runs on WebGPU, which Safari does not support.

The camera gate is off (`CAMERA_GATE_ENABLED = false` in both `controls.js` and `main.js`), so the system starts without waiting for you to pick a camera. Choose the camera in the Setup panel behind the cog icon.

The launchers

start.command

Single machine, standalone role. Sources `preflight.sh`, reads the LAN IP from the `en0` interface, opens Chrome, then runs `node server.js 3000`.

start-master.command

Same as above but starts the server with `--master`. Use it on the machine that holds the phone controller and drives the others. It prints the main page URL, the phone URL, the slave link address, and the LAN IP that the slave machines need. Write that IP down before you walk to the other machines.

start-slave.command

Starts the server with `--slave <master-ip>`. It asks for the master IP and offers a default of `192.168.1.100`. Press Return to accept the default, or type the real address. To change the default permanently, edit `DEFAULT_MASTER_IP` near the top of the file.

The slave connects upstream to `ws://<master-ip>:3000/slave-link` and retries every 3 seconds if the link drops, so you can start the slaves before the master and they will find it.

The dependency scripts

preflight.sh

Every launcher sources this before starting the server. It first clears the quarantine flag and restores the executable bit on the sibling `.command` and `.sh` files, then runs four checks and repairs what it can:

1. Node.js is installed and version 18 or newer.
2. The `ws` package resolves. If not, it runs `npm ci`, falling back to `npm install`.
3. Google Chrome is installed, in `/Applications` or `~/Applications`.
4. All ten files in `vendor/` exist and are the right size. If any are missing or truncated, it calls `./vendor.sh` to fetch them.

When it cannot fix something, it prints the problem, waits for you to press Return, and stops. It will not open Chrome on a half-working rig. Only step 4 needs an internet connection, and only when `vendor/` is incomplete.

You do not run this file yourself. It is sourced rather than executed, which is what lets it abort the launcher.

vendor.sh

Downloads every browser-side dependency into `vendor/`, about 12 MB: p5.js, the TensorFlow.js stack, the pose-detection wrapper, the qrcode library, and the MoveNet MultiPose Lightning weights. Nothing loads from a CDN at runtime, so the installation runs with no internet and does not break when an upstream package publishes a new version.

```
./vendor.sh           # fetch anything missing, skip what is already there
./vendor.sh --force   # re-download everything
```

Version numbers are pinned in variables at the top of the script. Change them there and nowhere else. `preflight.sh` calls this automatically, so you only run it by hand when you want to refresh or repair on purpose.

server.js

The HTTP server and WebSocket relay, both on port 3000. It serves the project directory, relays messages between every connected client, and answers two endpoints: `/api/role` reports whether this machine is standalone, master, or slave, and `/api/phone-url` returns the LAN address for the phone. The launchers start it for you.

Setting up more than one machine

One machine per projector. Parameters stay in sync across all of them.

1. Put all machines on the same network, wired if you can.
2. Start the master first with `start-master.command`. Note the IP it prints.
3. On each other machine, run `start-slave.command` and enter that IP.
4. Check the connection. `/api/role` on a slave reports `connected: true` once the link is up, and on the master it reports how many slaves have attached.

First run on a new machine

Clear the quarantine flag first, as described above. Then the first launch needs an internet connection so `vendor.sh` can download the browser dependencies and `npm` can install `ws`. Both are one-time. After that the machine runs offline.

If you are setting up on site with no network, copy the whole project folder including `vendor/` and `node_modules/` from a machine that already works. `preflight.sh` will find everything present and start straight away.

When the launch stops

The launcher will not open, or macOS says it cannot verify the developer. The quarantine flag is still set. Run `xattr -dr com.apple.quarantine .` in the project folder.

Double-clicking opens the file in a text editor. The executable bit is missing. Run `chmod +x *.command *.sh`.

"Node.js is not installed" or the version is too old. Install the LTS build from nodejs.org, or run `brew install node`.

"Google Chrome is not installed." The check looks in `/Applications` and `~/Applications`. Install Chrome from google.com/chrome.

"Could not download the browser dependencies." The machine is offline or the CDN is unreachable. Connect once and run `./vendor.sh`, then launch again.

The phone URL shows `localhost`. The launchers read the address from `en0`, which is the Wi-Fi interface on most Macs. On a wired connection `en0` may have no address. The QR code in the controls popup is more reliable, because `server.js` scans every interface for the first non-internal IPv4 address. Use the QR code, or run `ifconfig | grep 'inet '` to list the addresses on every interface and pick the one on your network.

No controls window. Chrome blocked the popup. Allow popups for `localhost:3000` and reload the main page. You can also open `controls.html` in a new tab by hand.

The phone will not connect. It has to be on the same network as the master, and the Terminal window has to still be open.

Keyboard shortcuts on the main page

`F` toggles fullscreen. `+` and `−` change the boid size.

Shutting down

Press Control-C in the Terminal window, or close it. Do this on every machine.